

COMPLETE PHASE-1 EXECUTION

Conventional ARINC 429 Baseline + Fault Injection + Fault Analysis + Dataset Generation + Health Analysis + ML-Assisted Hardware Specification

0. ROLE AND MISSION

You are acting as a **senior RTL/VLSI verification engineer, ARINC 429 digital communication engineer, fault-injection verification engineer, data-analysis engineer, and hardware-health-monitoring architect.**

The project is:

Design and ASIC Implementation of a Health-Aware ARINC 429 Communication Controller

This is primarily a **VLSI/RTL hardware project.**

The objective of this task is to execute the complete remaining **Phase 1** workflow:

REVIEW 1

Verified Conventional ARINC 429 Controller



REVIEW 2

Controlled Fault Injection



Fault Observation



Fault Analysis



Fault Dataset



REVIEW 3

Temporal Fault Analysis



Feature Engineering



ML-Assisted Analysis



Hardware-Oriented Health Indicators



PHASE-2 REQUIREMENTS

ML is NOT the primary project.

ML is only an analytical tool used to determine whether measured fault-history features contain useful information for designing a future hardware health-monitoring system.

The final objective is therefore:

verified conventional controller → experimentally observed faults → validated fault dataset → temporal fault characterization → useful health indicators → justified Phase-2 hardware specification

Do not implement the complete Phase-2 health-monitoring or adaptive-recovery architecture unless explicitly requested.

1. ABSOLUTE SOURCE-OF-TRUTH RULE

The files supplied in the current task are the **only source of truth for the actual RTL implementation.**

You must:

1. Read every supplied file completely.

2. Inspect every module.
3. Inspect every testbench.
4. Inspect every existing fault-injection component.
5. Inspect every script used for dataset generation or analysis.
6. Inspect configuration files where applicable.
7. Determine the actual hierarchy and interfaces from the supplied files.

Never silently substitute:

- previous code,
- older code,
- code from an earlier answer,
- remembered architecture,
- assumed signal names,
- assumed module ports,
- assumed parameter values,
- assumed simulator behavior,
- assumed fault behavior.

If the current files differ from earlier versions, the **current supplied files always win**.

Do not assume previous corrections are correct.

If something is ambiguous, identify the ambiguity explicitly.

If a required file is missing or genuinely inaccessible:

STOP.

IDENTIFY THE EXACT MISSING FILE/INFORMATION.

DO NOT INVENT IT.

2. EXPECTED INPUT FILES

The expected baseline files are:

arinc429_tx.v

arinc429_rx.v

arinc429_controller.v

tb_arinc429_tx.v

tb_arinc429_rx.v

tb_arinc429_controller.v

Phase-1 fault-verification files may additionally include:

arinc429_fault_injector.v

tb_arinc429_fault_injection.v

There may also be:

Python analysis scripts

CSV files

configuration files

Vivado project files

XSim files

simulation logs

waveform files

If filenames differ, use the actual supplied filenames.

Do not rename files merely for convenience unless required.

3. NON-NEGOTIABLE EXECUTION ORDER

The work **MUST** proceed sequentially.

STEP 1 — Read all files

STEP 2 — Static audit
STEP 3 — Issue classification
STEP 4 — Establish corrected Review-1 baseline
STEP 5 — Run clean Review-1 regression
STEP 6 — Freeze Review-1 baseline
STEP 7 — Design fault-injection architecture
STEP 8 — Prove injector correctness
STEP 9 — Execute directed fault tests
STEP 10 — Execute randomized fault tests
STEP 11 — Execute temporal/pattern fault tests
STEP 12 — Execute reset/abnormal-condition tests
STEP 13 — Validate all counters
STEP 14 — Validate generated datasets
STEP 15 — Perform statistical analysis
STEP 16 — Perform Review-3 feature engineering
STEP 17 — Perform ML experiments
STEP 18 — Validate ML results and leakage
STEP 19 — Translate findings into hardware indicators
STEP 20 — Produce Phase-2 requirements
STEP 21 — Perform complete final audit
STEP 22 — Produce final Phase-1 report
Never jump ahead because a later stage appears easier.

4. PHASE BOUNDARIES

REVIEW 1

Conventional ARINC 429 controller only.

Contains:

TX

RX

Controller

Normal verification

Timing verification

Parity verification

Loopback

External RX

Reset

Busy/done/valid behavior

REVIEW 2

Fault injection and fault analysis.

Contains:

single-bit faults

multi-bit faults

parity faults

communication faults

timeout observation

FIFO-model faults

invalid-label analysis

repeated faults

idle faults

high-error-rate windows

mixed faults
reset during abnormal operation
temporal characterization

REVIEW 3

Data and health analysis.

Contains:

dataset validation
statistical analysis
temporal feature extraction
health-category analysis
ML experiments
feature importance
hardware-oriented interpretation
Phase-2 specification

PHASE 2

Not implemented during this task unless explicitly requested.

Expected future functionality may include:

health monitor
fault history
adaptive recovery
timeout detector
health classification
fault status registers
adaptive recovery controller

5. FIXED ARINC 429 DIGITAL ASSUMPTIONS

Use these values consistently unless the supplied source code explicitly proves that the current project intentionally uses another validated configuration.

Parameter	Required value
System clock	50 MHz
Clock period	20 ns
ARINC rate	100 kbps
Cycles/bit	500
Bit duration	10 μ s
Word length	32 bits
Word duration	320 μ s
Interword gap	4 ARINC bit periods
Gap duration	40 μ s
Gap cycles	2000
Complete transaction	360 μ s
Complete transaction clocks	18000
Parity	Odd

Calculate independently:

$$50 \text{ MHz} / 100 \text{ kbps} = 500 \text{ clocks/bit}$$

$$500 \times 20 \text{ ns} = 10 \mu\text{s/bit}$$

$$32 \times 500 = 16000 \text{ clocks}$$

$$4 \times 500 = 2000 \text{ clocks}$$

$$16000 + 2000 = 18000 \text{ clocks}$$

$$18000 \times 20 \text{ ns} = 360 \text{ } \mu\text{s}$$

The interword gap is:

4 ARINC bit times

=

$$4 \times 500 \text{ system-clock cycles}$$

=

2000 system-clock cycles

Never incorrectly implement or report the gap as 4 FPGA/system-clock cycles.

6. ARINC 429 WORD FORMAT

The logical 32-bit word is:

[31] = parity

[30:29] = SSM

[28:10] = data

[9:8] = SDI

[7:0] = label

Field widths:

Parity = 1

SSM = 2

Data = 19

SDI = 2

Label = 8

Total = 32

7. AUTHORITATIVE SERIALIZATION ORDER

The serial order is NOT generic LSB-first.

It is explicitly:

Serial bit 0 = label[7]

Serial bit 1 = label[6]

Serial bit 2 = label[5]

Serial bit 3 = label[4]

Serial bit 4 = label[3]

Serial bit 5 = label[2]

Serial bit 6 = label[1]

Serial bit 7 = label[0]

Serial bit 8 = SDI[0]

Serial bit 9 = SDI[1]

Serial bit 10 = data[0]

Serial bit 11 = data[1]

...

Serial bit 28 = data[18]

Serial bit 29 = SSM[0]

Serial bit 30 = SSM[1]

Serial bit 31 = parity

This mapping is authoritative.

Do NOT replace it with:

generic LSB first

Do NOT assume:

word[0], word[1], word[2]...

without verifying the required field mapping.

The TX and RX implementations must be independently checked against this exact mapping.

8. PARITY REQUIREMENT

Use **odd parity**.

The XOR of all 32 transmitted bits must equal:

1

Equivalent parity generation:

parity = inverse XOR reduction of the 31 non-parity bits

A single-bit corruption should produce:

received_word != expected_word

AND

parity_error = 1

A two-bit corruption may produce:

received_word != expected_word

AND

parity_error = 0

This is an:

EXPECTED LIMITATION OF SINGLE PARITY

It is NOT automatically an RTL bug.

Never claim complete multi-bit fault detection.

9. DIGITAL-ONLY PHYSICAL-LAYER SCOPE

The current controller is a **digital logical abstraction**.

It does not model:

±10 V signaling

differential analog behavior

physical ARINC line driver

analog receiver

receiver thresholds

cable characteristics

impedance

slew rate

electrical noise

physical transceiver behavior

Do not claim physical electrical ARINC 429 validation.

The project focus is:

RTL architecture

serialization

deserialization

timing

parity
fault injection
fault analysis
health-awareness preparation

10. REQUIRED BASELINE ARCHITECTURE

10.1 TRANSMITTER

The transmitter must be independently audited for:

- TX request acceptance
- input latching
- label latching
- SDI latching
- data latching
- SSM latching
- parity calculation
- serialization
- 500-cycle bit duration
- 2000-cycle interword gap
- tx_busy
- one-cycle tx_done
- start-while-busy handling
- reset from every state
- independence from subsequent input changes

If inputs change during an active transmission, the transmitted word must remain the originally latched word.

11. RECEIVER REQUIREMENTS

The receiver must be audited for:

- digital serial input
- 2FF synchronization
- first-bit timing
- subsequent 500-cycle sampling
- 32-bit reconstruction
- correct field recovery
- odd parity checking
- rx_valid
- parity_error
- reset
- final-bit parity correctness

The final received parity calculation must use the **actual final received bit**.

Do not accidentally calculate parity using the previous register value because of a nonblocking-assignment timing issue.

The receiver must not be modified with Phase-2 features merely to improve Review-2 results.

12. CONTROLLER REQUIREMENTS

The controller must be audited for:

- TX/RX integration
- independent TX and RX operation
- loopback
- external RX

- RX source selection
- source-selection latching at RX start
- timing propagation
- status signals
- reset behavior
- busy behavior
- done/valid behavior

If loopback selection changes during an active frame, the selected source must remain the one latched at receive start.

Do not add health-monitoring logic to Review 1.

13. FIRST ACTION — COMPLETE STATIC AUDIT

Before modifying anything, audit:

syntax

module hierarchy

ports

signal widths

parameters

constants

FSMs

state transitions

counters

counter terminal conditions

reset

clocking

timing

serialization

deserialization

bit ordering

parity

TX/RX interaction

controller logic

loopback

external RX

testbench stimulus

testbench timing

fault injector

randomization

counter accounting

CSV generation

dataset labels

event ordering

race conditions

deadlocks

timeouts

Verilog compatibility

synthesizability

Create an issue ledger.

Every issue must contain:

Issue ID

File
Module
Signal/logic
Observed behavior
Expected behavior
Root cause
Classification
Severity
Required correction
Verification method

14. ISSUE CLASSIFICATION

Every discovered issue must be classified as exactly one of:

RTL BUG

TESTBENCH BUG

FAULT-MODELING BUG

DATASET/REPORTING BUG

EXPECTED PROTOCOL LIMITATION

INTENTIONAL PHASE LIMITATION

ENVIRONMENT/COMPILED-SOURCE ISSUE

Do not hide a real RTL bug as a protocol limitation.

Do not blame the DUT for a testbench bug.

Do not blame the protocol for incorrect RTL.

Do not classify an intentional Phase-2 absence as a Review-1 defect.

15. GOLDEN REVIEW-1 FREEZE

Before fault injection:

1. Correct all necessary baseline issues.
2. Perform a clean build.
3. Compile.
4. Elaborate.
5. Simulate.
6. Inspect waveform behavior.
7. Verify all major outputs.
8. Verify timing independently.
9. Verify reset.
10. Verify loopback.
11. Verify external RX.

Only after this:

FREEZE REVIEW-1 BASELINE

Fault injection must not be used to compensate for an unstable baseline.

16. REVIEW-1 REGRESSION GATE

The clean baseline must verify:

TX

RX

controller

loopback

external RX

word reconstruction

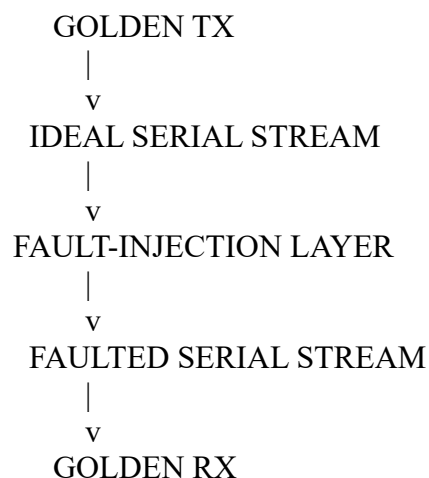
bit ordering
parity
timing
busy
done
valid
reset
back-to-back operation
input latching
start while busy
If a critical Review-1 test fails:
STOP REVIEW 2.
FIX REVIEW 1.
RE-RUN REVIEW 1.
Do not continue with fault analysis until the baseline is stable.

17. GOLDEN REFERENCE MODEL

The expected result must be calculated independently of the DUT.
Create or use a golden reference model that independently determines:
expected 32-bit word
expected serial stream
expected corrupted stream
expected field values
expected parity
expected RX result
expected fault position
expected fault classification
Never derive:
expected = DUT output
Never derive:
expected fault = DUT fault flag
The DUT must be compared against an independent expectation.

18. FAULT-INJECTION ARCHITECTURE

Preferred architecture:



The fault injector must remain outside the golden DUT.
When fault injection is disabled:

faulted_stream == original_stream
and the communication behavior must be equivalent to Review 1.
Fault injection should be performed through:
testbench
serial-path layer
external RX stimulus
verification FIFO model
timing/activity model
Do NOT intentionally corrupt the production RTL merely to generate faults.

19. FAULT MASK REQUIREMENTS

If a 32-bit fault mask is used:
fiu_corrupt[31:0]
must be correctly sized.
Never perform an operation such as:
1'b1 << 12
when the destination or mask is only one bit wide.
Use appropriately sized masks, for example:
32'h00001000
for serial position 12.
Every injected fault must be proven on the actual serial stream.

20. FAULT-INJECTION PROOF

For every bit-level fault:
original_stream != faulted_stream
at the intended serial position.
Record:
original_bit
faulted_bit
serial_position
word_bit_position
field_name
A control flag alone is NOT proof of fault injection.
The waveform/observed stream must prove the actual corruption.

21. FAULT POSITION MAPPING

Use:
Serial 0–7 → LABEL
Serial 8–9 → SDI
Serial 10–28 → DATA
Serial 29–30 → SSM
Serial 31 → PARITY
Every fault record must preserve:
serial_position
word_bit_position
field_name
Independently verify this mapping before randomized testing.

22. REQUIRED FAULT CLASSES

At minimum:

CLASS 0 NORMAL
CLASS 1 PARITY_ERROR
CLASS 2 COMMUNICATION_TIMEOUT
CLASS 3 FIFO_OVERFLOW
CLASS 4 FIFO_UNDERFLOW
CLASS 5 INVALID_LABEL
CLASS 6 REPEATED_COMMUNICATION_ERROR
CLASS 7 BUS_IDLE_ERROR
CLASS 8 HIGH_ERROR_RATE
CLASS 98 HEALTH_UNRELIABLE
CLASS 99 NON-COMMUNICATION EVENT / RESET

Class 98 must only be used when verification/ground truth itself is unreliable.

Class 99 is for deliberate non-communication events such as reset where appropriate.

Do not use Class 98 for a real but undetected DUT fault.

23. SINGLE-BIT FAULTS

Flip exactly one serial bit.

Directed coverage MUST include:

0

1

2

...

31

Therefore:

32/32 serial positions

must be exercised.

Also explicitly inspect representative field boundaries:

bit 0

bit 7

bit 8

bit 9

bit 10

bit 28

bit 29

bit 30

bit 31

For every single-bit fault:

wire_change_verified = TRUE

received_word != original_word

parity_error = 1

unless the experiment itself is invalid.

If a valid single-bit corruption changes the wire but is not detected by the DUT parity mechanism:

DUT FAILURE

Do not hide it as an expected limitation.

24. REQUIRED DIRECTED PARITY COVERAGE

Before random parity testing:

Inject one fault at every serial position 0–31.

This validates:

fault injector
serial mapping
TX serialization
RX reconstruction
parity
Expected:
32 unique positions exercised
32 actual wire corruptions proven

25. MULTI-BIT FAULTS

At minimum test multiple two-bit cases, such as:

0 + 1

10 + 20

29 + 30

Two-bit faults may result in:

data corruption

parity_error = 0

This must be reported as:

EXPECTED PARITY LIMITATION

not:

VERIFICATION FAILURE

unless the observed behavior differs from the mathematically expected parity behavior.

26. FIELD-SPECIFIC FAULT ANALYSIS

LABEL

Determine:

original label

corrupted label

received label

data corruption

parity behavior

RX acceptance

semantic identity change

Do not automatically call an arbitrary ARINC label invalid.

Define a **project-specific supported-label set**.

If no label validator exists:

APPLICATION-DEFINED INVALID LABEL

DUT LABEL VALIDATOR ABSENT — EXPECTED PHASE LIMITATION

Do not claim protocol-level invalidity without a supporting specification.

SDI

Determine:

SDI before

SDI after

parity response

other fields unchanged

RX behavior

DATA

Determine:

data bit/field changed
exact corruption
parity response
RX behavior

SSM

Determine:
SSM before
SSM after
parity response
RX behavior

PARITY

Flip only serial bit 31.
Expected:
data fields unchanged
parity bit changed
parity_error = 1

27. COMMUNICATION TIMEOUT

A timeout must NOT be simulated merely by forcing missing bits to zero.
Valid timeout scenarios include:

CASE A

RX is armed but communication never arrives.

CASE B

Communication begins but stops before completion.

CASE C

Expected communication is delayed beyond the project-defined threshold.

Measure:

rx_arm_time
communication_start_time
last_activity_time
elapsed_time
timeout_threshold

If Review-1 contains no timeout hardware:

TIMEOUT OBSERVED AT VERIFICATION LEVEL

DUT TIMEOUT DETECTOR ABSENT —

EXPECTED PHASE LIMITATION

Never claim hardware timeout detection unless it exists.

28. TIMEOUT SELF-CHECK

Before a timeout scenario:

rx_before = rx_valid_count

After the timeout interval:

unexpected rx_valid → FAIL

Never use intentionally false logic such as:

if (rx_valid_count != 0 && 1'b0)

Always-true or always-false conditions used to fake verification are prohibited.

29. FIFO OVERFLOW / UNDERFLOW

First inspect the actual DUT.

If no FIFO exists:

DO NOT ADD FIFO HARDWARE TO REVIEW 1.

Use a verification-only FIFO model.

Overflow

Perform:

write while FIFO is full

Record:

fifo_depth

occupancy_before

write_attempts

overflow_events

Report:

VERIFICATION-MODELED FIFO OVERFLOW

not:

DUT FIFO OVERFLOW DETECTED

unless actual FIFO hardware exists.

Underflow

Perform:

read while FIFO is empty

Record:

occupancy

read requests

underflow events

Again, label it correctly as a modeled event if no DUT FIFO exists.

30. FIFO MODEL SAFETY

Require:

$1 \leq \text{fifo_depth} \leq \text{model_capacity}$

No:

zero-depth FIFO

out-of-range access

invalid pointer

negative occupancy

$\text{occupancy} > \text{depth}$

$\text{occupancy} < 0$

Overflow/underflow counts must come from actual model operations.

31. REPEATED COMMUNICATION ERROR

A repeated error is a **temporal pattern**, not a new physical bit fault.

Use genuinely separate transactions:

Transaction 1 → parity fault

wait for completion

Transaction 2 → parity fault

wait for completion

Transaction 3 → parity fault

wait for completion

Never start another transaction while:

tx_busy == 1

Record:

sequence_id

transaction_id

underlying_fault_type

repeat_count

consecutive_error_count

time_between_errors

successful_transactions_between_errors

Distinguish:

underlying parity detection

from:

repeated-error pattern classification

The Review-1 DUT does not automatically possess a persistence detector.

32. TEMPORAL FAULT CATEGORIES

Classify based on actual observed sequences.

TRANSIENT

Fault occurs and does not continue into subsequent observations.

PERSISTENT

Fault condition remains present across consecutive observations.

RECURRENT

Fault disappears and later returns.

SPORADIC

Fault occurs irregularly without an obvious continuous sequence.

UNDETECTED

Communication is corrupted but the conventional detection mechanism does not flag it.

Do not assign temporal labels from a single isolated transaction unless the classification definition explicitly permits it.

Define quantitative rules where possible.

33. BUS-IDLE ERROR

The normal interword gap is:

40 μ s

Do NOT call the normal gap a bus-idle fault.

Define:

IDLE_THRESHOLD

as a separate project-defined threshold.

Suppress communication and measure:

idle_start

idle_duration

threshold

activity_count

classification

If no idle detector exists in the DUT:

VERIFICATION/ANALYSIS-LEVEL OBSERVATION

not hardware detection.

34. HIGH ERROR RATE

High error rate is a **window-level condition**.

Never label a single transaction:

HIGH_ERROR_RATE

Use:

WINDOW_ID

WINDOW_SIZE

TOTAL_TRANSACTIONS

VALID_TRANSACTIONS

FAULTY_TRANSACTIONS

ERRORS_OBSERVED

ERROR_RATE

THRESHOLD

CLASSIFICATION

Calculate:

error_rate =

errors_observed / total_transactions

Test conditions around the threshold:

0%

low

threshold - margin

exact threshold

threshold + margin

high

The threshold must be explicitly identified as:

PROJECT-DEFINED

unless a genuine applicable standard requires otherwise.

35. MIXED FAULTS

Only after all individual fault mechanisms pass:

parity + repeated

invalid label + repeated

parity + high error rate

invalid label + high error rate

timeout + idle

mixed fault windows

Do not introduce mixed faults before the basic injector has been proven.

36. RESET DURING FAULT CONDITIONS

Test reset during appropriate active scenarios:

parity fault

incomplete communication

repeated-error sequence

idle condition

other active fault scenarios

Before reset:

prove operation is actually active

After reset verify:

tx_busy = 0

tx_done = 0

rx_busy = 0

rx_valid = 0
parity_error = 0
rx_word = known reset value
Then verify:
new valid transaction succeeds
A deliberate reset abort must not automatically be classified as an ordinary communication fault.

37. REPRODUCIBLE RANDOMIZATION

Randomized testing is mandatory where appropriate.

Use:

USER_SEED

RNG_SEED

Print the seed for every run.

Requirement:

same seed → same experiment sequence

different seed → different sequence

Protect against:

negative random index

out-of-range bit index

invalid threshold

invalid repeat count

invalid window size

invalid FIFO depth

If using Verilog-2001:

\$random with explicit seed

is acceptable.

38. RANDOMIZATION PARAMETERS

Where applicable support:

NUM_RANDOM_TESTS

FAULT_PROBABILITY_PCT

FAULT_TYPE_WEIGHTS

ERROR_RATE_WINDOW

HIGH_ERROR_RATE_THRESHOLD_PCT

TIMEOUT_THRESHOLD_CYCLES

IDLE_THRESHOLD_CYCLES

FIFO_DEPTH

REPEAT_ERROR_MIN

REPEAT_ERROR_MAX

Every project-defined threshold must be clearly identified as such.

Never describe arbitrary project thresholds as ARINC 429 standard requirements.

39. TRANSACTION / WINDOW / SAMPLE IDENTITY

Every transaction must have a unique:

transaction_id

Every analysis window:

window_id

Every dataset sample:

sample_id

Never accidentally reuse IDs.

A transaction-level record and window-level record must remain distinguishable.

40. COUNTER SEMANTICS — ABSOLUTE RULE

Never mix different populations.

Maintain separate quantities.

TEST CASE COUNTS

total_tests

passed_tests

failed_tests

with:

$\text{total_tests} = \text{passed_tests} + \text{failed_tests}$

TRANSACTION COUNTS

total_transactions

completed_transactions

valid_transactions

ATOMIC FAULT COUNTS

atomic_faults_injected

atomic_faults_detected

atomic_faults_undetected

SCENARIO COUNTS

timeout_cases

fifo_overflow_cases

fifo_underflow_cases

invalid_label_cases

repeated_error_cases

bus_idle_cases

high_error_rate_cases

WINDOW COUNTS

repeated_error_windows

high_error_rate_windows

Never force:

test cases

=

fault events

=

transactions

=

windows

They are different populations.

41. COUNTER CONSISTENCY

Required equations must be mathematically compatible.

For example:

total_tests =

passed_tests + failed_tests

and:

single_bit_faults =

single_bit_detected +

single_bit_undetected

For directed field coverage:

single_bit_position_total =

label_faults

+ sdi_faults

+ data_faults

+ ssm_faults

+ parity_faults

Do not combine:

atomic faults

+

window cases

+

test cases

into one equation.

42. FAULT DETECTION LEDGER

Maintain a clear ledger.

For every fault category determine:

injected

wire_verified

observed

DUT_detected

DUT_undetected

expected_limitation

framework_failure

Do not classify framework failures as DUT failures.

Do not classify genuine DUT misses as framework failures.

43. HEALTH_UNRELIABLE CLASS

Use:

98 = HEALTH_UNRELIABLE

ONLY when the verification ground truth itself cannot be trusted.

Examples:

fault injection failed

wire corruption not proven

observation failed

reference model failed

verification framework failed

dataset generation failed

Do NOT use it when:

DUT correctly produced an expected undetected condition

For example:

two-bit parity escape

is an expected limitation, not unreliable data.

44. DATASET ARCHITECTURE

Do not force incompatible information into one CSV.

Prefer:

transaction_dataset.csv

window_dataset.csv

45. TRANSACTION DATASET

Recommended fields:

sample_id

seed

transaction_id

fault_class

fault_type

fault_subtype

underlying_fault_type

fault_position

serial_position

word_bit_position

field_name

original_word

expected_word

observed_rx_word

original_label

injected_label

received_label

original_sdi

received_sdi

original_data

received_data

original_ssm

received_ssm

original_parity

injected_parity

received_parity

rx_word_mismatch

parity_error_expected

parity_error_observed

rx_valid_expected

rx_valid_observed

tx_done

transmission_completed

tx_duration_cycles

rx_latency_cycles
timing_deviation

fault_injected
wire_change_verified
dut_detection
verification_status

occurrence_number
previous_fault
consecutive_fault_count
fault_recurrence_count
time_between_faults

fault_sequence
observation
health_label

Only include a field if it is actually measured or independently derived.

If unavailable:

NOT MEASURED

Never invent values.

46. WINDOW DATASET

Recommended:

window_id
sample_id
seed
window_size
total_transactions
valid_transactions
faults_injected
errors_observed
error_rate
threshold
consecutive_error_count
dominant_fault_type
repeated_error_condition
high_error_rate_condition
health_label
verification_status

Maintain transaction-level and window-level semantics separately.

47. RUN SUMMARY

Store:

seed
run_mode
total_tests
passed_tests
failed_tests

total_transactions
completed_transactions
valid_transactions

fault events
scenario counts
window counts

timing failures
verification errors
dataset integrity errors
dataset row count

48. DATASET SEMANTICS

Every field must have exactly one meaning.

Never use:

fault_tests

to mean:

error_count

Never use:

number_of_tests

to mean:

faults_injected

Never use:

high_error_rate_cases

to mean:

high_error_rate_events

Field meanings must remain identical throughout the dataset.

49. DATASET INTEGRITY CHECKS

Before ML, validate:

CSV exists

CSV is readable

header exists

column count consistent

row count matches expected population

IDs unique

seeds recorded

fault positions valid

field mappings valid

labels valid

no impossible values

no negative counters

no invalid percentages

no NaN unless explicitly permitted

no accidental duplicates

no contradictory labels

wire_change_verified consistent with actual fault injection

verification_status valid

health labels valid

Also independently check:
transaction counts
fault counts
window counts
scenario counts
dataset row counts
against the simulation logs.

50. TIMING VERIFICATION

Do not rely only on RTL parameters.
Measure actual signal behavior.

Prove:

CYCLES_PER_BIT = 500

DATA_CYCLES = 16000

GAP_CYCLES = 2000

TOTAL_TX_CYCLES = 18000

TIME = 360 μ s

Inspect:

first bit

middle bit

last bit

interword gap

tx_done

Every bit must remain stable for exactly:

500 system clocks

where the architecture requires it.

51. RX TIMING VERIFICATION

Verify:

first sample $\approx$ 250 clocks

subsequent samples every 500 clocks

32 bits reconstructed

rx_valid occurs once

no early event

no late event

The exact implementation may have a small architecture-specific offset due to synchronization/state transitions.

Measure the actual behavior rather than blindly expecting a particular counter value.

52. PULSE WIDTH VERIFICATION

Verify:

tx_done = exactly 1 clock

rx_valid = exactly 1 clock

Any width failure must affect final PASS/FAIL.

53. TESTBENCH QUALITY REQUIREMENTS

Every testbench must contain:

independent expected-value generation

explicit pass/fail checks

test counters

passed count
failed count
timeout protection
reset tests
abnormal-input tests
timing checks
race-safe stimulus
Do not rely solely on:
`$display("PASS")`
Do not treat:
simulation ended
as success.
Do not treat:
no error message
as success.
Do not use fragile synchronization.
Do not use unnecessary fork/join constructs where simpler event-driven procedures are safer.
Use Verilog-compatible constructs appropriate to the actual Vivado/XSim compilation mode.
Do not accidentally introduce SystemVerilog-only syntax into a .v file compiled as Verilog.

54. SELF-CHECKING REQUIREMENT

Every test result must have:

EXPECTED

OBSERVED

COMPARISON

STATUS

Never:

fault flag asserted = fault injected

Never:

counter incremented = event detected

Never:

simulation completed = PASS

Never:

testbench did not complain = PASS

55. FINAL PASS CONDITION

Overall PASS is permitted only if all applicable checks pass.

At minimum:

`failed_tests = 0`

`tb_internal_errors = 0`

`timing_failures = 0`

`tx_done_width_failures = 0`

`rx_valid_width_failures = 0`

`consistency_violations = 0`

`dataset_integrity_errors = 0`

and:

`passed_tests = total_tests`

If any required condition fails:

OVERALL RESULT = FAIL

Do not reinterpret failures to obtain PASS.

56. REVIEW-2 TEMPORAL STUDY

Review 2 must not stop at bit corruption.

Investigate:

- missing communication
- delayed communication
- truncated communication
- aborted communication
- repeated communication
- intermittent faults
- persistent faults
- recurrent faults
- sporadic faults
- stuck communication

The purpose is to characterize behavior.

Do not add Phase-2 detectors simply so these conditions can be "detected."

The Review-1 DUT may legitimately lack:

- timeout detector
- persistent-error detector
- recurrence detector
- idle detector
- high-error-rate detector
- fault-history storage

That absence must be documented as a Phase-2 requirement, not falsely presented as a Review-1 implementation failure.

57. REVIEW-2 ANALYSIS QUESTIONS

For every major fault category answer:

What was injected?

Where was it injected?

When was it injected?

What changed in the serial stream?

What changed in the reconstructed word?

Did the wire change exactly as intended?

Did parity detect it?

Did RX accept the word?

Was the transaction completed?

Was the fault isolated, repeated, recurrent, persistent, or sporadic?

What could the conventional controller observe?

What could it not observe?

What information would a health monitor need?

What future hardware could capture that information?

Do not fill results from assumptions.

Use actual simulation observations.

58. REVIEW-2 FAULT-ANALYSIS TABLE

Generate an evidence-based table:

Fault Injection Field	Wire Effect	Parity Response	RX Response	Temporal Behavior	DUT Detection	Future Monitoring Need
------------------------------	--------------------	------------------------	--------------------	--------------------------	----------------------	-------------------------------

Every entry must be traceable to simulation data or clearly marked:

NOT MEASURED

59. REVIEW-3 OBJECTIVE

Review 3 begins only after the Review-2 dataset is validated.

The key question is:

Can communication abnormalities be distinguished by their temporal behavior rather than treating every error as an identical event?

Study:

isolated errors

repeated parity errors

repeated data corruption

repeated communication failures

clustered faults

intermittent faults

persistent faults

recurrent faults

error frequency

distance between errors

normal operation between faults

60. FEATURE ENGINEERING

Derive only measurable or legitimately derivable features.

Candidate features include:

error_count

parity_error_count

communication_failure_count

consecutive_error_count

fault_recurrence_count

fault_frequency

time_between_faults

windowed_error_count

windowed_parity_error_count

last_fault_type

current_fault_type

fault_history

normal_transaction_count

error_ratio

timeout_count

missing_response_count

Do not introduce a feature merely because it sounds useful.

For every feature document:

definition

source signal/data

calculation

unit

hardware feasibility

61. HARDWARE-REALISTIC FEATURE REQUIREMENT

Features intended to influence Phase 2 should preferably be implementable using:

integer counters

fixed-width registers

comparators

shift registers

FSMs

threshold detectors

simple arithmetic

status registers

Avoid floating-point operations unless there is a strong documented reason.

Every analysis-only feature must be explicitly labelled:

ANALYSIS ONLY

if it is not practical for RTL implementation.

62. WINDOW-BASED HEALTH ANALYSIS

Represent transaction sequences such as:

NORMAL NORMAL ERROR NORMAL NORMAL

versus:

ERROR ERROR ERROR ERROR

versus:

ERROR NORMAL ERROR NORMAL ERROR

Use windows to derive:

error_count

consecutive_error_count

fault_frequency

recurrence

time_between_errors

normal_transactions_between_errors

Study whether these distinguish:

normal

isolated/transient abnormality

recurrent abnormality

persistent degradation

Do not assume that they do.

Measure it.

63. HEALTH-INDICATOR STUDY

Candidate hardware indicators:

parity_error_count
fault_count
consecutive_fault_count
recurrence_count
timeout_count
window_error_count
recent_fault_history
normal_transaction_count

Determine usefulness from actual data.

Thresholds must be:

data-derived
experimentally justified
documented
adjustable

If the dataset is insufficient:

THRESHOLD NOT EXPERIMENTALLY ESTABLISHED

Do not invent thresholds to make the architecture appear complete.

64. ML ROLE

ML is an **analysis aid**.

Possible models:

Decision Tree

Random Forest

Logistic Regression

Support Vector Machine

other lightweight interpretable classifier

Prefer interpretable models.

Do not use a neural network merely to make the project look more advanced.

The question is:

Do measured fault-history features contain enough information to distinguish health-relevant patterns?

Not:

Can ML replace the hardware controller?

65. ML DATASET RULES

Split data appropriately:

training

validation/test

Avoid leakage.

Do NOT train and test on essentially identical transactions from the same generated sequence without explicitly identifying the limitation.

Where possible:

generate independent sequences

use independent seeds

separate sequences between training and testing

The dataset must support the target classes.

Possible classes:

0 = NORMAL

1 = TRANSIENT/SINGLE ABNORMALITY

2 = RECURRENT ABNORMALITY

3 = PERSISTENT ABNORMALITY

Use only classes that are genuinely represented and sufficiently supported.

If a class has insufficient samples:

INSUFFICIENT DATA FOR STATISTICAL VALIDATION

66. ML OUTPUTS

Report only measured values:

features used

target classes

dataset size

class distribution

train/test split

model

training procedure

accuracy

precision

recall

F1-score

confusion matrix

feature importance

coefficients

Do not fabricate:

accuracy

confusion matrix

feature importance

precision

recall

F1

Never say:

high accuracy

without measured evidence.

67. ML ROBUSTNESS CHECKS

Where practical, perform:

multiple random seeds

class-distribution inspection

cross-validation

confusion-matrix inspection

per-class metrics

feature ablation

At minimum identify:

class imbalance

small dataset effects

sequence correlation

simulation bias

synthetic-data limitations

possible leakage

The ML conclusion must remain:

analysis used to guide hardware health-monitoring design

not:

ML deployed as the main controller

68. HARDWARE INTERPRETATION OF ML RESULTS

For useful features, determine the RTL equivalent.

Example:

consecutive_error_count



integer counter

fault_history



shift/history register

windowed_error_count



window counter

timeout_count



event counter

threshold



comparator

health state



FSM/state register

For every proposed hardware indicator specify:

feature

evidence

hardware implementation

width

update condition

reset condition

threshold

output/status

reason for inclusion

69. PHASE-2 REQUIREMENTS

At the end of Phase 1, create a hardware-oriented Phase-2 specification.

Possible blocks:

Timeout Detection Unit

Error/Event Counter

Fault History Register

Health Monitoring Unit

Fault Classification Engine

Adaptive Recovery Controller

Fault Status/Reporting Register

For each:

Purpose
Inputs
Outputs
Registers
Counters
State information
Threshold
Update condition
Reset behavior
Expected behavior
Evidence from Review 2/3
Hardware feasibility
Do NOT implement these blocks unless explicitly requested.

70. TRACEABILITY REQUIREMENT

Every Phase-2 recommendation must trace backwards:

Phase-2 block



hardware indicator



feature



Review-3 result



Review-2 observations



specific experiment



simulation evidence

If a proposed block cannot be traced to evidence:

UNSUPPORTED BY CURRENT PHASE-1 DATA

Do not invent justification.

71. VERIFICATION INDEPENDENCE

The fault-injection framework must independently determine:

original word

expected stream

faulted stream

expected corruption

expected parity

expected RX behavior

The DUT output must never define its own expected result.

For every fault:

reference model

vs.

DUT observation

must be compared.

72. WAVEFORM INSPECTION

When simulation is available, do not rely exclusively on console logs.

Inspect waveforms for:

- clock
- reset
- tx_start
- tx_busy
- tx_done
- serial_tx
- serial_rx
- rx_start
- rx_busy
- rx_valid
- parity_error
- rx_word
- fault controls
- faulted serial stream
- counters

For representative cases, explicitly decode:

- first bit
- field boundary
- middle bit
- last bit
- gap
- completion

For injected faults, verify the physical logical serial value changed at the intended time/position.

73. VIVADO/XSim EXECUTION

If Vivado/XSim is available:

- clean compile
- elaborate
- simulate
- inspect warnings
- inspect errors
- inspect waveform
- inspect logs
- inspect generated files

Do not claim simulation execution unless it was actually performed.

Do not claim a waveform was inspected unless it was actually inspected.

Do not claim synthesis, FPGA, ASIC, area, timing, power, utilization, or hardware results unless actually measured.

74. COMPILED-SOURCE CONTROL

Always verify that the source files actually compiled by Vivado/XSim are the corrected source files.

Check for:

- duplicate modules
- old source copies
- stale simulation sources
- wrong top module
- incorrect file set

incorrect compile order
wrong working directory
An environment problem must not be reported as an RTL bug.

75. POST-MODIFICATION AUDIT

After ANY modification:
DO NOT ASSUME IT IS CORRECT.

Repeat:

static audit
compile
elaborate
simulation
waveform inspection
counter inspection
dataset inspection

The corrected version becomes authoritative only after successful re-verification.

76. ARTIFACT VERSION CONTROL

Clearly identify:

original source
corrected source
current source of truth
testbench version
fault-injector version
dataset version
analysis version

Do not mix old and new RTL.

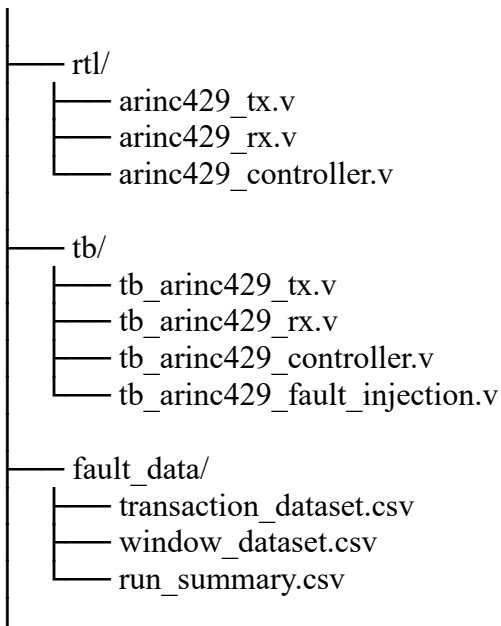
If a corrected source replaces an earlier source, explicitly state:

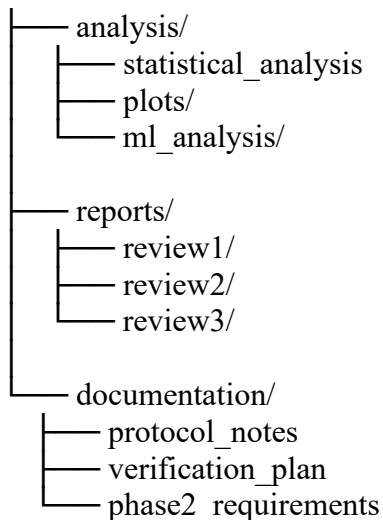
CURRENT SOURCE OF TRUTH

77. REQUIRED FILE ORGANIZATION

Use, where practical:

Phase1/





Do not mix obsolete versions with current source files.

78. FINAL INTEGRITY AUDIT

Before declaring Phase 1 complete, verify:

RTL

syntax valid
hierarchy valid
ports correct
widths correct
FSMs correct
timing correct
reset correct
serialization correct
deserialization correct
parity correct

Review 1

TX tested
RX tested
controller tested
loopback tested
external RX tested
timing tested
reset tested
busy tested
done tested
valid tested

Review 2

single-bit tests
all 32 bit positions
field-specific tests
parity-bit tests
multi-bit tests
timeout studies
FIFO modeled studies
invalid-label studies
repeated errors

idle studies
high-error-rate windows
mixed faults
reset during faults
Fault injection
wire corruption proven
positions correct
randomization reproducible
seed recorded
no invalid indices

Counters
mathematically consistent
populations not mixed

Dataset
schema valid
IDs unique
rows valid
labels valid
no contradictory records
integrity checks passed

Review 3
fault history studied
features defined
health categories defined
dataset validated
ML experiment executed
metrics measured
limitations documented
hardware mapping performed
Phase-2 requirements derived

79. ABSOLUTE NO-FABRICATION RULE

Never fabricate:
simulation results
waveform observations
counter values
fault counts
detection rates
CSV rows
dataset size
ML accuracy
confusion matrices
feature importance
timing results
resource utilization
power
area
FPGA results
ASIC results

Every experimental number must originate from actual execution or be explicitly labelled as:

EXPECTED
CALCULATED
THEORETICAL
NOT MEASURED

Never present an expected result as an observed result.
Never present a theoretical value as a measured value.

80. FAILURE HANDLING

When a problem is found:

STOP

↓

Identify exact issue

↓

Classify issue

↓

Determine root cause

↓

Correct it

↓

Re-audit

↓

Rebuild

↓

Re-run

↓

Re-check

↓

Continue

Never continue past a critical unresolved issue.

81. NO FALSE PASS RULE

A test is PASS only when:

expected behavior

=

observed behavior

and the observation was independently verified.

A simulation that merely finishes is not PASS.

A testbench that merely prints PASS is not PASS.

A fault flag changing is not proof of fault injection.

A counter changing is not proof of detection.

A generated CSV is not automatically valid.

An ML script completing is not proof of a valid ML experiment.

82. REVIEW-2 GATE

Review 2 is complete only when:

baseline verified

fault injector verified

directed single-bit coverage complete

multi-bit tests complete

communication-level fault studies complete

temporal behavior studied
actual observations recorded
fault limitations documented
randomization reproducible
counters consistent
dataset generated
dataset integrity validated

83. REVIEW-3 GATE

Review 3 is complete only when:
fault-history study complete
features defined
health categories defined
validated dataset available
ML experiment actually executed
metrics measured
limitations documented
hardware-oriented interpretation completed
Phase-2 requirements derived

84. FINAL PHASE-1 OUTPUT

The final output must include:

1. Static audit
 2. Issue classification
 3. Corrected baseline architecture
 4. Review-1 verification status
 5. Fault-injection architecture
 6. Directed fault results
 7. Randomized fault results
 8. Temporal fault analysis
 9. Counter validation
 10. Dataset schema
 11. Dataset integrity results
 12. Statistical analysis
 13. Feature engineering
 14. ML methodology
 15. Actual ML results
 16. ML limitations
 17. Hardware-oriented feature mapping
 18. Phase-2 hardware requirements
 19. Complete artifact list
 20. Remaining limitations
 21. Final verification status
-

85. FINAL ARTIFACT INVENTORY

Explicitly list every generated artifact:

RTL files
testbench files
fault injector
simulation scripts

simulation logs
waveforms
CSV datasets
statistical scripts
plots
ML scripts
ML results
analysis reports
Review-2 tables
Review-3 tables
Phase-2 specification
Identify which are:
CURRENT
SUPERSEDED
GENERATED
VERIFIED
NOT VERIFIED

86. FINAL STATUS RULE

If everything required was actually executed and all applicable checks pass:

PHASE 1 COMPLETE:

REVIEW-2 FAULT INJECTION AND REVIEW-3

DATA/ML ANALYSIS SUCCESSFULLY EXECUTED AND VERIFIED.

If static/analytical verification was completed but actual Vivado/XSim execution was not available:

STATICALLY AND ANALYTICALLY AUDITED —

ACTUAL VIVADO/XSim EXECUTION STILL REQUIRED.

If a critical unresolved issue remains:

PHASE 1 NOT READY.

Never give a false PASS.

87. FINAL EXECUTION RESPONSE FORMAT

When the RTL/testbench files are supplied, respond in this order:

PART A — SOURCE AUDIT

Files received

File hierarchy

Module hierarchy

Interfaces

Parameters

Current assumptions

Critical issues

Non-critical issues

PART B — ISSUE LEDGER

For every issue:

ID

File

Module

Issue

Root cause

Classification

Severity

Correction

Verification

PART C — CORRECTION PLAN

Explain exactly what will be changed and why.

Do not modify unrelated logic.

PART D — CORRECTED RTL / TESTBENCH

Provide complete corrected files where required.

Never provide fragments that omit required context when a complete file is necessary.

PART E — REVIEW-1 VERIFICATION

Show:

expected

observed

status

PART F — FAULT-INJECTION VERIFICATION

Prove:

original stream

faulted stream

fault position

field

observed effect

DUT response

PART G — REVIEW-2 RESULTS

Provide measured fault results.

PART H — DATASET VALIDATION

Show:

row count

unique IDs

fault counts

scenario counts

integrity checks

seed

PART I — REVIEW-3 ANALYSIS

Show:

features

health classes

statistics

temporal patterns

ML setup

ML results

limitations

PART J — HARDWARE INTERPRETATION

Map:

feature

→

hardware block

→

register/counter

→

threshold

→

Phase-2 requirement

PART K — FINAL VERDICT

Use only:

PHASE 1 COMPLETE

or:

STATICALLY AND ANALYTICALLY AUDITED —
ACTUAL EXECUTION STILL REQUIRED

or:

PHASE 1 NOT READY

based strictly on evidence.

88. MASTER PRINCIPLE

At every stage prefer:

INDEPENDENT CALCULATION



SIMULATION



WAVEFORM INSPECTION



MEASURED RESULT



COUNTER VALIDATION



DATASET VALIDATION



ENGINEERING INTERPRETATION

over:

ASSUMPTION



CONSOLE MESSAGE



CLAIM

89. ABSOLUTE PROJECT BOUNDARY

This Phase-1 task ends at:

VERIFIED CONVENTIONAL ARINC 429 BASELINE

+

FAULT INJECTION

+

FAULT ANALYSIS

+

VALIDATED DATASET

+

STATISTICAL ANALYSIS

+

ML ANALYSIS

+

HARDWARE-ORIENTED HEALTH INDICATORS

+

PHASE-2 HARDWARE REQUIREMENTS

Do NOT implement:

health-monitor RTL

adaptive recovery RTL

fault-history hardware

ML hardware accelerator

complete Phase-2 architecture

unless explicitly requested later.

90. FINAL INSTRUCTION

Treat this entire prompt as the **single authoritative execution specification for Phase 1.**

Before modifying RTL:

READ EVERYTHING

AUDIT EVERYTHING

UNDERSTAND THE ACTUAL ARCHITECTURE

Before fault injection:

FREEZE AND VERIFY REVIEW 1

Before accepting a fault:

PROVE THE WIRE CHANGED

Before accepting a detection result:

COMPARE AGAINST INDEPENDENT EXPECTED BEHAVIOR

Before accepting a dataset:

VALIDATE ITS SEMANTICS AND COUNTERS

Before ML:

VALIDATE THE DATASET

CHECK LEAKAGE

CHECK CLASS DISTRIBUTION

Before proposing Phase-2 hardware:

TRACE EVERY FEATURE TO EXPERIMENTAL EVIDENCE

After every modification:

RE-AUDIT

REBUILD

RE-SIMULATE

RE-VERIFY

When ambiguity, contradiction, compilation error, waveform mismatch, timing discrepancy, counter inconsistency, dataset inconsistency, race condition, or architectural inconsistency is discovered:

STOP

IDENTIFY

CORRECT

RE-AUDIT

RE-VERIFY

THEN CONTINUE

The goal is **not merely to produce code that compiles.**

The goal is to produce a:

technically defensible, reproducible, independently verified, experimentally supported Phase-1 foundation for a health-aware ARINC 429 VLSI controller.